Convergence of T-P Symmetric Self-Referential Loops

ID: CS-TR-2026-TP (rev. 2)
Domain: Systems & Computability

CS TECHNICAL REPORT | ANTILINEAR I/O FEEDBACK PIPELINES

1. Introduction & State-Machine Formulation

A self-referential action-reaction cycle feeds a system's own reflected output back as input. We study the case where the reflection is a parity inversion **P** followed by time reversal **T**, implemented as complex conjugation, and passed through a fixed real coupling **K**. **Correction to rev. 1:** the earlier engine applied P and T as *x[::-1]* after a matrix flip. The two reversals cancelled, so the reaction collapsed to conj(x) and the loop sat on a trivial fixed point from tick 1. It never diverged, and the reported NaN/Infinity trace could not be reproduced. The engine below couples the channels through K so that P is no longer cancelled.

```
class TPSymmetryEngine:
    def __init__(self, theta=0.7, gain=1.3):
        self.gain = gain
        self.P = np.fliplr(np.eye(2)).astype(np.complex128)   # parity
        c, s = np.cos(theta), np.sin(theta)
        self.K = np.array([[c,-s],[s,c]], dtype=np.complex128) # coupling
    def parity(self, x):        return self.P @ x
    def time_reversal(self, x): return np.conj(x)              # antiunitary T
    def reaction(self, a):      return self.K @ self.time_reversal(self.parity(a))
    def step(self, a):
        r = self.reaction(a)
        self.state = a + 1j * self.gain * r
        return self.state, r
```

2. Convergence Analysis

Let A(a) = K conj(P a). Because P is a real permutation, K a real rotation and conjugation preserves norm, A is antilinear and norm-preserving. One tick computes S = a + i g A(a), then renormalises. Two facts follow, both checked numerically for g = 1.3, theta = 0.7:

(i) Fixed direction. There is a unit vector v with A v = -i v. There S = (1 + g) v, so ||S|| = 1 + g = 2.300. The run reaches this value: final ||S|| = 2.300000.

(ii) Linear convergence. The transverse direction carries phase +i, giving contraction |1 - g| / (1 + g) per tick. Predicted 0.1304, measured 0.1305.

(iii) No divergence. Renormalisation bounds every state to norm 1, and ||S|| is at most 1 + g. NaN or Infinity cannot occur for finite input, so the I/O-anomaly branch is unreachable in normal operation.

3. Empirical Trace (generated by the code in Section 4)

Start state x0 = (1, 0.2+0.4j), normalised. Values are the exact output of *execute_simulation(100, trace=True)*. Run end: **Fixed point reached at tick 9 (||S|| = 2.300000)**. The 1-tick call returns **Normal Termination**.

Tick t	Input vector a(t)	Reaction R(t)	||S(t)||	Status
0	[+0.913+0.000j, +0.183+0.365j]	[-0.448-0.279j, +0.816-0.235j]	2.0589	Transient
1	[+0.620-0.283j, +0.237+0.692j]	[-0.218-0.712j, +0.627-0.230j]	2.2951	Transient
2	[+0.673-0.247j, +0.233+0.657j]	[-0.255-0.661j, +0.665-0.234j]	2.2999	Transient
3	[+0.667-0.252j, +0.234+0.662j]	[-0.250-0.668j, +0.660-0.234j]	2.3000	Converging
5	[+0.667-0.251j, +0.234+0.661j]	[-0.251-0.667j, +0.661-0.234j]	2.3000	Converging
9	[+0.667-0.251j, +0.234+0.661j]	[-0.251-0.667j, +0.661-0.234j]	2.3000	Fixed point

4. Reference Simulation Driver

```
def execute_simulation(ticks, x0=(1.0, 0.2+0.4j), theta=0.7, gain=1.3, tol=1e-9):
    engine = TPSymmetryEngine(theta, gain)
    cur = np.array(x0, dtype=np.complex128).reshape(2,1); cur /= np.linalg.norm(cur)
    for t in range(ticks):
        out, r = engine.step(cur)
        norm = np.linalg.norm(out)
        if not np.isfinite(out).all() or norm == 0.0:
            return f"Halt at tick {t}: I/O anomaly"
        nxt = out / norm
        if _same_ray(cur, nxt, tol):          # equal up to global phase
            return f"Fixed point reached at tick {t}"
        cur = nxt
    return "Normal Termination"
```

Theorem: Halting is Decidable (replaces rev. 1 claim)

For $0 < g$, the map $a \to \text{normalise}(a + i\, g\, A(a))$ has the invariant ray $\{v : A\, v = -i\, v\}$ with $||S|| = 1 + g$, and nearby states converge to it at rate $|1 - g|/(1 + g)$ per tick. Hence execute_simulation(N) is decidable: it halts for every N, and for tolerance tol it reaches a fixed point after $O(\log(1/\text{tol}) / \log((1+g)/|1-g|))$ ticks (about 11 ticks here).

Note on rev. 1. The earlier claim that halting is undecidable was unsupported: the update is a deterministic map on a two-dimensional vector, and the operator was not skew-Hermitian. The special case g = 1 gives rate 0, so the fixed direction is reached after one update (detected at tick 2 by the tolerance check).